AI / ML • NETWORK SECURITY PROJECT • ASSESSMENT

# ML-Based Network Intrusion Detection

*Abstract · Methodology · Simulation Results*
*A controlled benchmark of five classifiers on the corrected CICIDS2017 dataset*

Ahmad Azim & Pawan Kumar • Computer Networks

# Abstract

*What we built, what we measured, and what we found*

Signature-based intrusion detection only catches attacks it already has a rule for. This project benchmarks five machine-learning classifiers — Logistic Regression, Random Forest, XGBoost, RBF-SVM and an MLP — for flow-based intrusion detection on the corrected CICIDS2017 dataset, all trained and tested under one leakage-safe pipeline with three class-imbalance strategies.

Beyond accuracy we measure statistical significance, per-flow latency, SHAP explanations, and generalisation: to a time-separated split, to attack families held out of training, and to an independent dataset (CSE-CIC-IDS2018).

**Result:** tree ensembles reach 0.999 macro-F1 but are statistically tied; every model misses attack variants it has not seen, and Random Forest / XGBoost detect only 0.3% of 2018 attacks. Near-perfect benchmark scores do not transfer to a new network.

**1.71M**
labelled flows after cleaning (from 2.10M)

**5 × 3**
classifiers × class-imbalance strategies

**0.999**
best macro-F1 (random split)

**0.3%**
2018 attacks caught by RF / XGBoost

METHODOLOGY

# End-to-End Pipeline

*Six stages; every model sees exactly the same data, split, features and metrics*

1

## Data

Corrected CICIDS2017: re-extracted with a fixed CICFlowMeter and relabelled; 5 days, 2.1M flows

2

## Clean

Drop IPs, ports, flow IDs; remove NaN/inf; de-duplicate before splitting; cap benign at 600k

3

## Split

80/20 stratified random split, plus a 5-minute time-block split as a leakage check

4

## Features

82 → 60 after correlation pruning (|r| > 0.95) → 30 via RFE with a Random Forest, train split only

5

## Train

5 classifiers × {none, class weights, SMOTE}; RandomizedSearchCV with a fixed budget

6

## Evaluate

Per-class metrics, FPR, PR-AUC, significance tests, SHAP, latency, generalisation tests

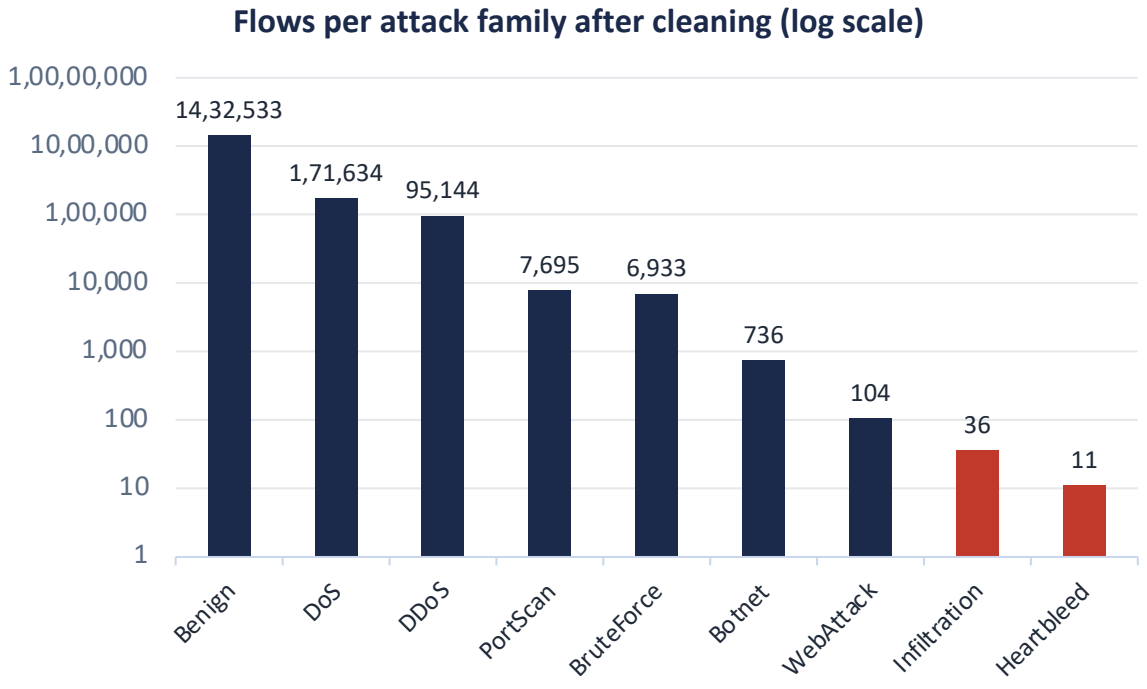
**Tools:** Python 3.12 • pandas • scikit-learn • XGBoost • imbalanced-learn • SHAP • ONNX Runtime • one config file + `ids` CLI, fixed seed 42

METHODOLOGY

# Dataset & Preprocessing

*Corrected CICIDS2017: cleaning removes 18% of flows, almost all of them duplicates*

| | |
|---|---|
| Flows loaded (Mon–Fri) | 2,099,976 |
| NaN / infinite values | −5 |
| Duplicates with conflicting labels | −2,678 |
| Exact duplicates | −382,467 |
| **After cleaning** | **1,714,826** |
| After capping benign at 600k | 882,246 |
| **Train / test (80 / 20)** | **705,796 / 176,450** |




**Port scans collapse from 231k to 7.7k flows** once ports are dropped: scan probes are identical SYN→RST flows. **Infiltration (36) and Heartbleed (11)** are too small for multi-class evaluation; they stay in the binary and leave-one-attack-out tests.

METHODOLOGY

# Classifiers & Class-Imbalance Handling

*Five model families, each trained with three imbalance strategies (14 runs; MLP has no class weights)*

1

### Logistic Regression

*Linear baseline*

**Tuned:** C (regularisation)
**Input:** log + standardise

2

### Random Forest

*Bagged trees*

**Tuned:** trees, depth, split size, max features
**Input:** raw features

3

### XGBoost

*Gradient boosting*

**Tuned:** learning rate, depth, trees, subsampling
**Input:** raw features

4

### RBF-SVM

*Margin classifier*

**Tuned:** C, γ — trained on 60k rows (cost ∝ $n^2$)
**Input:** log + standardise

5

### MLP

*Feed-forward net*

**Tuned:** layer sizes, learning rate, α, activation
**Input:** log + standardise

IMBALANCE STRATEGIES

### None

Learn from the natural class mix.

### Class weights

Loss weighted by inverse class frequency (sample weights for XGBoost).

### SMOTE

Inside each CV fold only; minority classes raised to 20k rows; neighbours found in log space.

METHODOLOGY

# Evaluation Protocol

*Accuracy alone hides failures; each test below answers a different question*

### 1 Metrics

Macro-F1, per-class recall, benign false-positive rate, PR-AUC, detection rate at 0.1% and 1% FPR

### 2 Significance

Nadeau–Bengio corrected t-test on 5-fold CV scores, plus McNemar's test on the test set

### 3 Leakage check

Time-block split: flows grouped into 5-minute blocks that never straddle train and test

### 4 Explainability

SHAP TreeExplainer: global feature ranking and per-flow explanations of alerts

### 5 Deployability

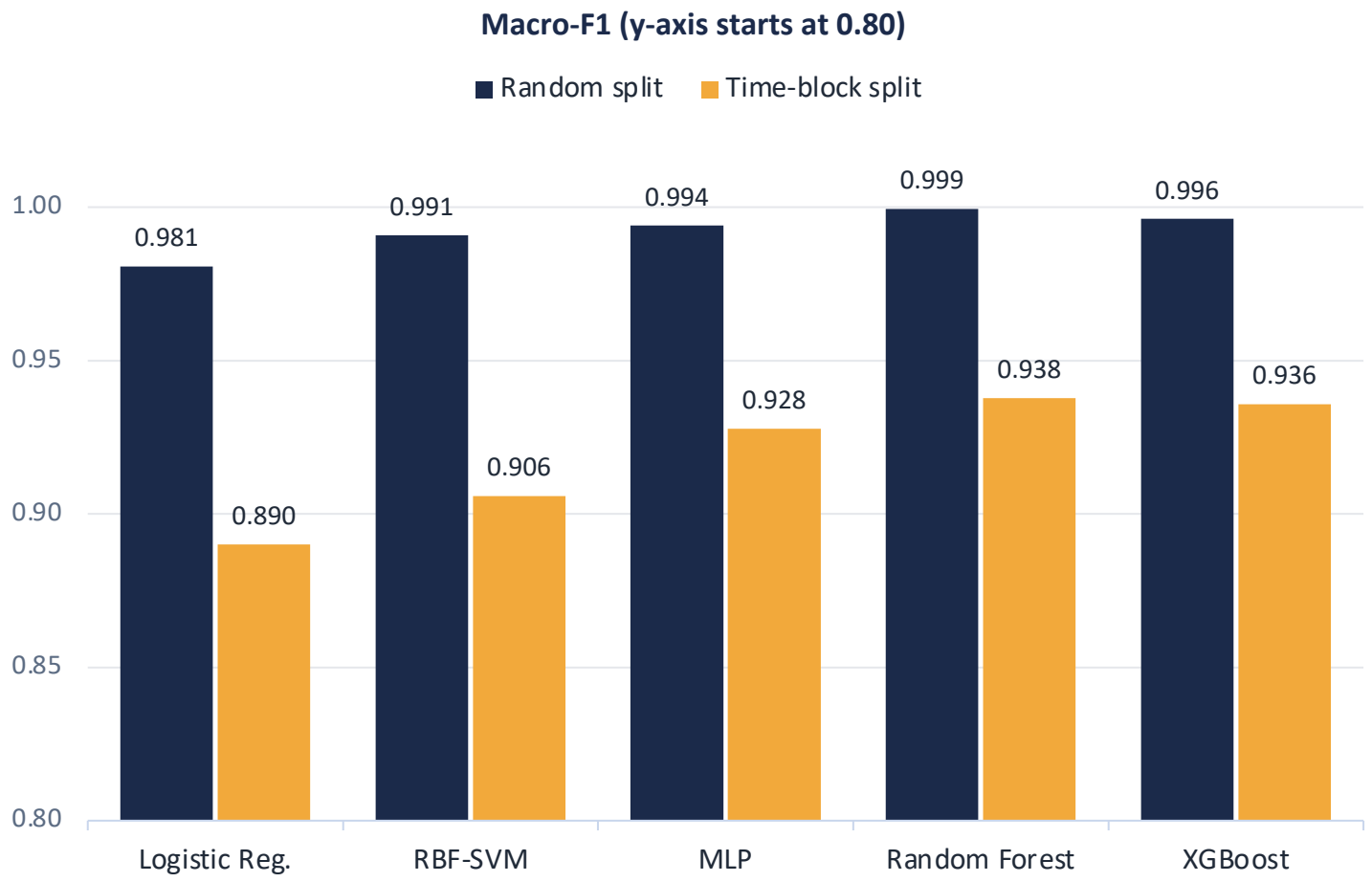
Model size, single-flow latency (1 thread), batch throughput, ONNX Runtime latency

### 6 Generalisation

Leave-one-attack-out (zero-day proxy) and corrected CSE-CIC-IDS2018 with no retraining

# Overall Performance

*Best strategy per model; macro-F1 on the held-out test set*




**Random Forest ≈ XGBoost**

Statistically tied: corrected t-test p = 0.92 (random) and 0.96 (time-block).

**Benign vs attack barely moves**

Time-block split: attack recall ≥ 0.996, benign FPR < 0.01% for Random Forest.

**Multi-class drop: 0.99 → 0.94**

Almost entirely one class (WebAttack). See the next slide.

SIMULATION RESULTS

# Per-Class Recall & Class Imbalance

*Time-block split, best strategy per model; aggregate accuracy hides the weak class*

| Model | Benign | Botnet | BruteForce | DDoS | DoS | PortScan | WebAttack |
|---|---|---|---|---|---|---|---|
| **Logistic Reg.** | 0.999 | 1.000 | 0.995 | 0.999 | 0.997 | 0.741 | **0.455** |
| **RBF-SVM** | 1.000 | 1.000 | 0.997 | 1.000 | 0.982 | 0.966 | **0.455** |
| **MLP** | 1.000 | 1.000 | 0.996 | 1.000 | 0.982 | 0.966 | **0.455** |
| **Random Forest** | 1.000 | 1.000 | 0.998 | 1.000 | 0.982 | 0.983 | **0.455** |
| **XGBoost** | 1.000 | 1.000 | 0.998 | 1.000 | 0.997 | 0.983 | **0.455** |
| Test flows | 121,112 | 162 | 1,982 | 46,293 | 8,847 | 58 | 22 |

*Cell colour: ≥ 0.99 blue, 0.90–0.99 light amber, 0.70–0.90 amber, < 0.70 red. Hypothesis H2 (low rare-class recall before SMOTE) is not supported on the corrected, de-duplicated data.*

### Unseen variant → missed

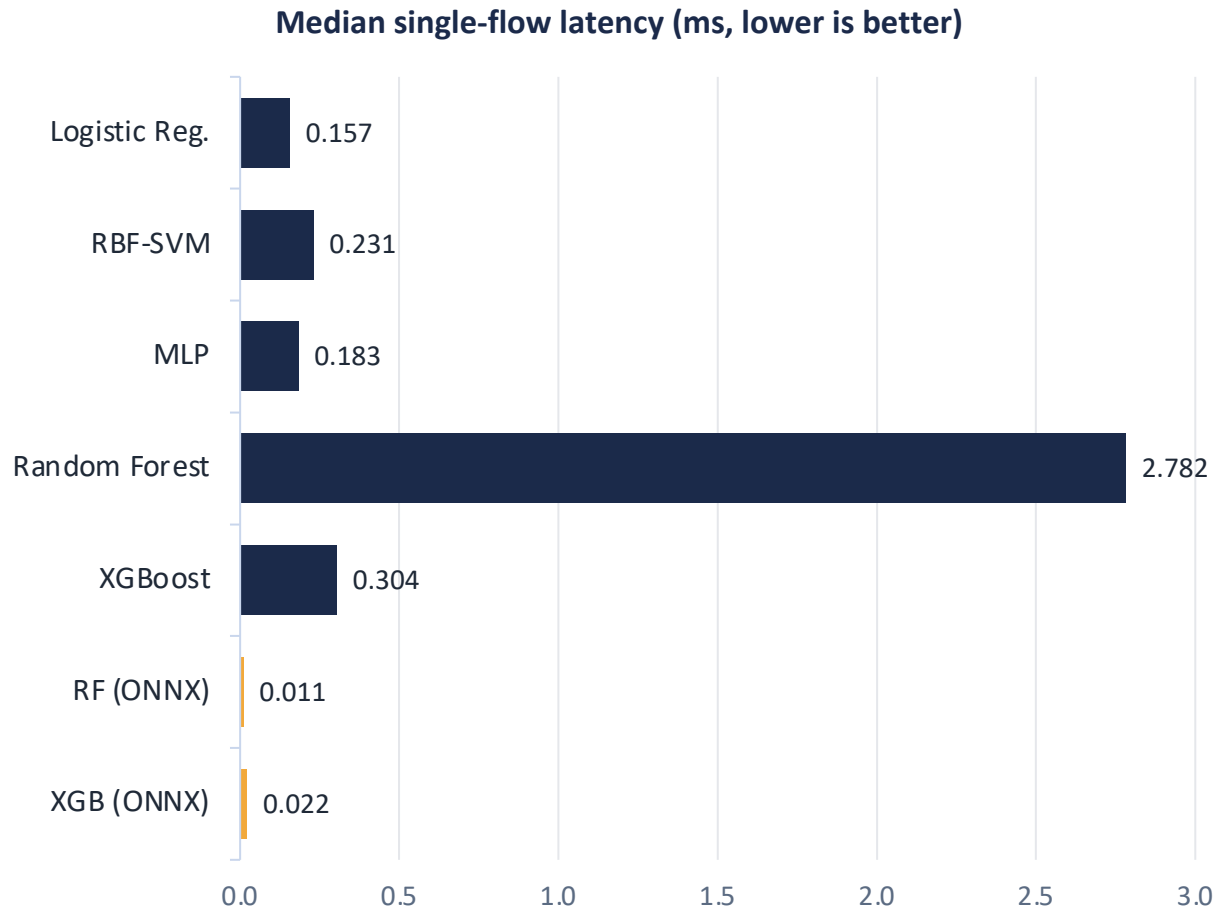
The time-block split put 12 of 13 SQL-injection flows in test. Every model missed all 12 (called benign or DoS); web brute force was caught 10/10.

### Rebalancing does not help

SMOTE gives no significant gain. Class weights hurt linear models: Logistic Regression macro-F1 0.98 → 0.79, benign FPR 0.12% → 2.3%.

# Deployability

*Cost of classifying one flow, measured single-threaded on an Apple M1 (8 GB)*



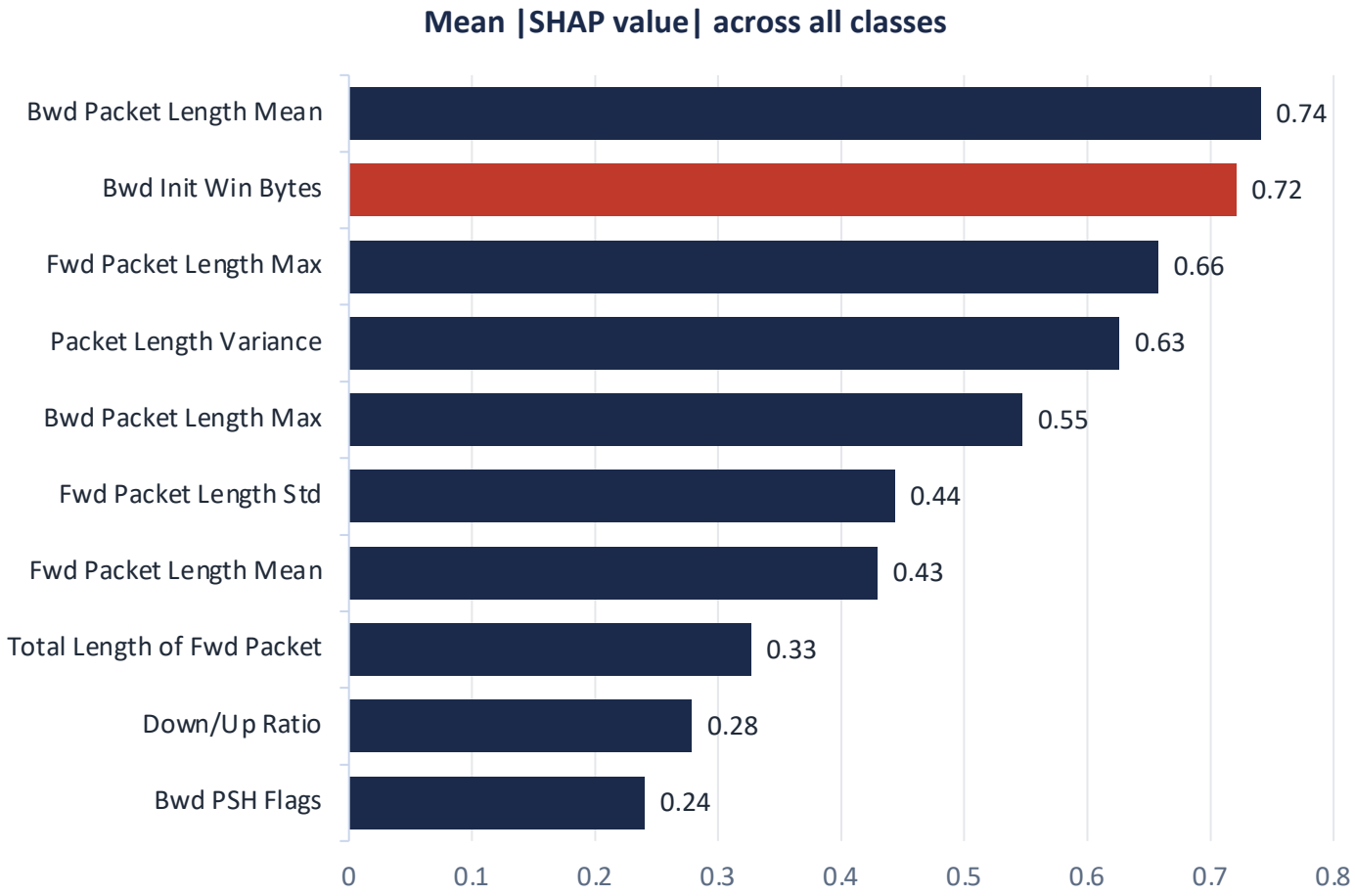

| Model | Size | Train | Flows / s |
|---|---|---|---|
| **Logistic Reg.** | 3 KB | 14 s | 3,534k |
| **RBF-SVM** | 106 KB | 1 s | 23k |
| **MLP** | 148 KB | 15 s | 1,525k |
| **Random Forest** | 2.7 MB | 54 s | 367k |
| **XGBoost** | 857 KB | 30 s | 274k |

### XGBoost: best accuracy-to-cost trade-off

Ties Random Forest on F1 at 9× lower latency and ~3× smaller. SVM is fast per flow but slowest in batch (~20k flows/s). Real detection delay is dominated by flow completion (up to a 120 s timeout).

# Explainability (SHAP)

*What drives XGBoost's decisions: top-10 features by mean |SHAP value|*




### Packet sizes dominate

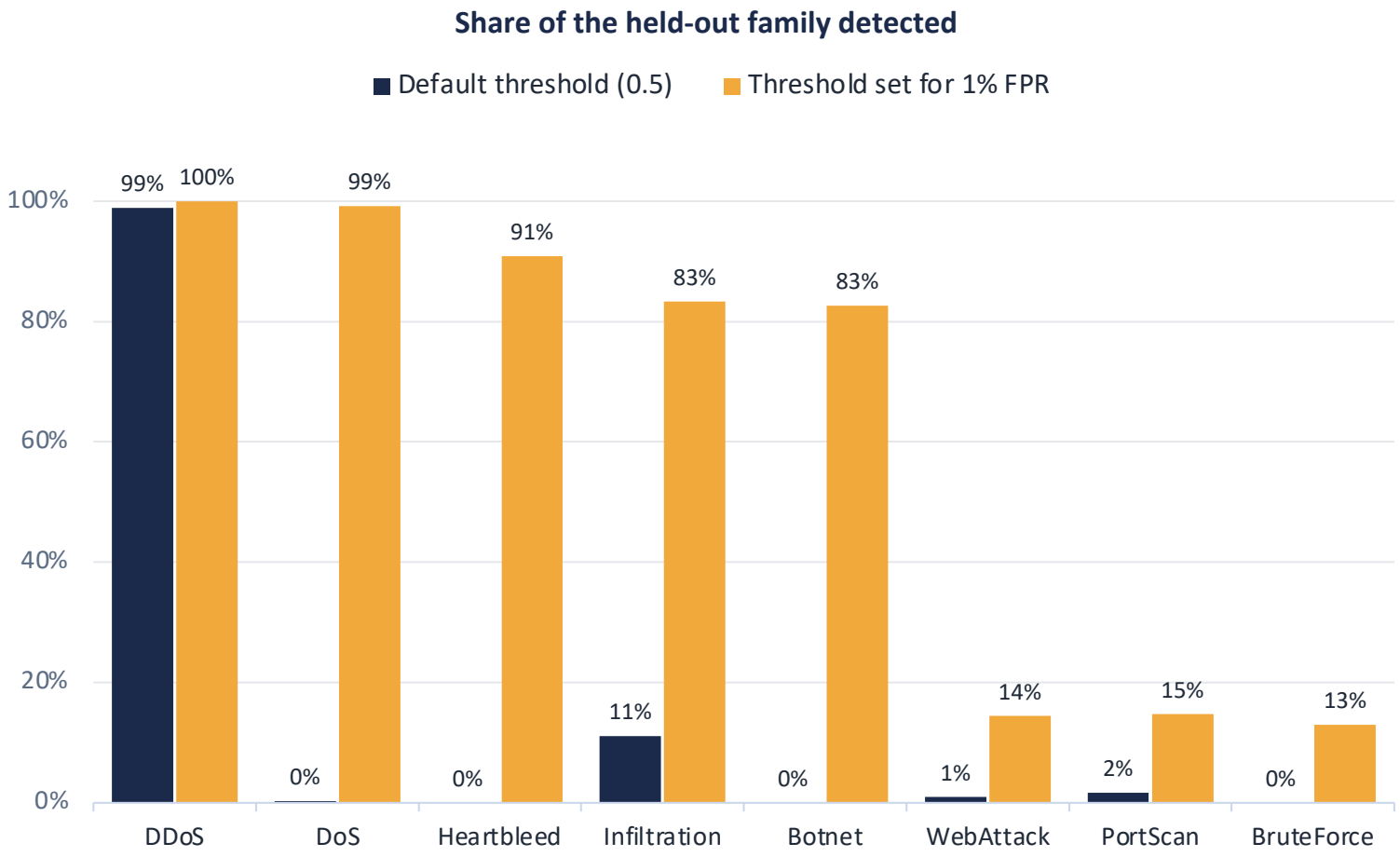
Backward/forward packet length statistics lead. This is defensible: DoS floods, scans and web attacks have characteristic request and response sizes.

### Red flag: Bwd Init Win Bytes is #2

The initial TCP window reflects the host operating system and testbed, not the attack. A model leaning on it learns the lab, which matches the cross-dataset collapse.

# Unseen Attacks: Leave-One-Attack-Out

*Binary XGBoost trained without one attack family, then tested on that family*




**Default threshold: ~0%**

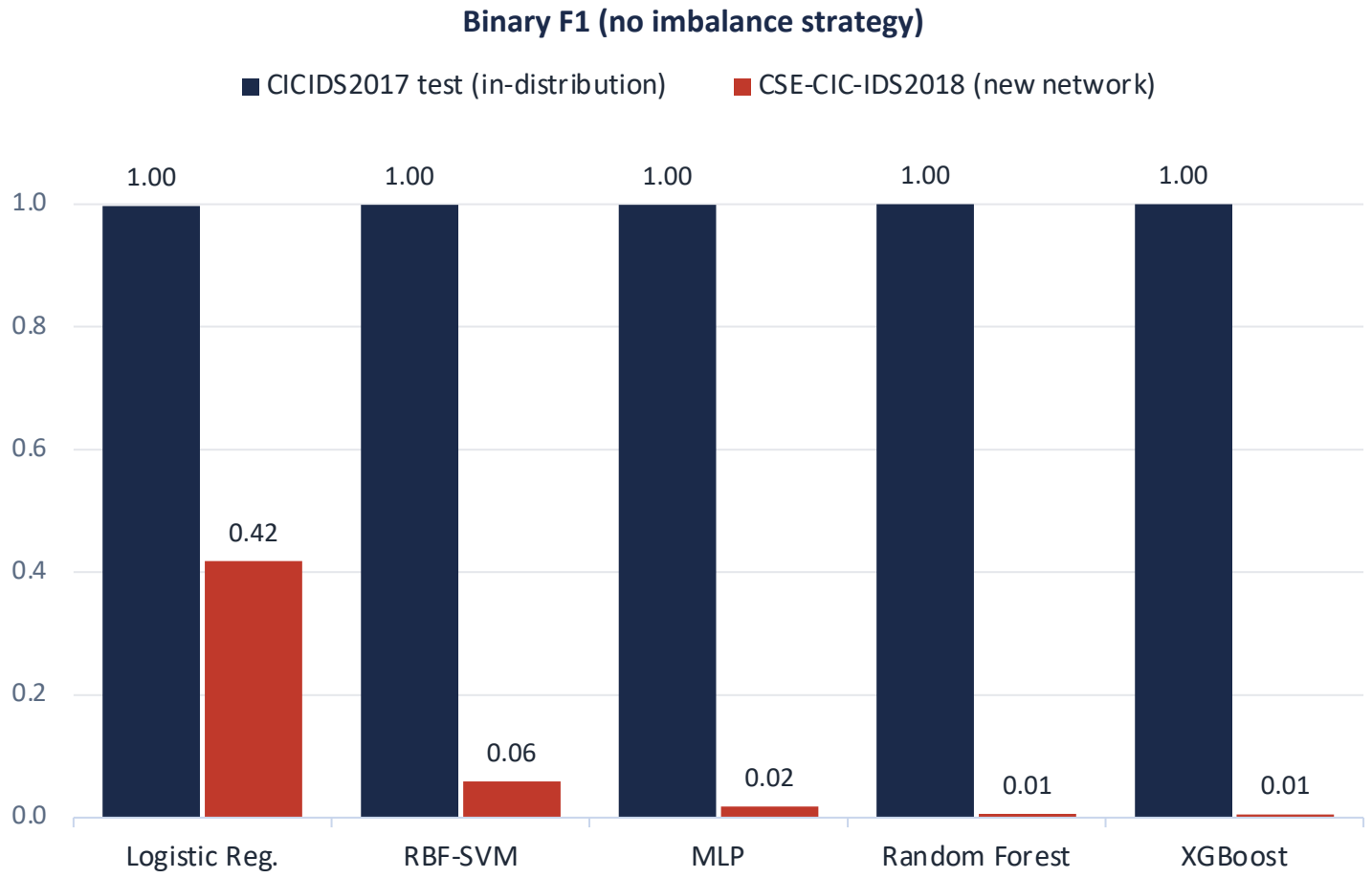
Except DDoS (similar to DoS), held-out families are almost never flagged; the zero-day claim fails as deployed.

**Signal is partly there**

Scores still rank unseen flows above benign (ROC-AUC 0.96–1.00). Re-tuning the threshold recovers DoS, Botnet and Infiltration, but not port scans, brute force or web attacks.

# Cross-Dataset Generalisation

*Trained on CICIDS2017, tested with no retraining on corrected CSE-CIC-IDS2018 (benign vs attack)*




**0.3%**

of 2018 attacks detected by Random Forest / XGBoost (in-distribution F1 0.9998)

### Simplest model transfers best

Logistic Regression keeps F1 0.42 (recall 27%, ROC-AUC 0.83). Even at a 1% FPR threshold no model exceeds 22% recall: this is distribution shift, not a threshold problem.

# Conclusions

**H1** — **Ensembles beat the linear baseline**
RF / XGBoost rank top, but the gap to MLP / SVM is not significant
**Partly supported**

**H2** — **Rare classes weak before SMOTE**
Rare-class recall already high; SMOTE adds nothing significant
**Not supported**

**H3** — **RF / XGBoost best accuracy–latency**
XGBoost: RF-level F1 at 9× lower latency; SVM cost untested (capped)
**Partly supported**

**H4** — **Accuracy drops on a new dataset**
RF / XGBoost 2018 F1 ≤ 0.01; detection 0.3%
**Strongly supported**

*“Near-perfect benchmark scores measure how well a model learns the lab, not how well it protects a network.”*

## Recommendation

Deploy XGBoost (via ONNX) for attacks seen in training: 0.99+ F1, ~0.02 ms per flow.

Do not take benchmark accuracy to a new network: retrain or adapt per site.

Drop testbed-specific features such as Init Win Bytes and re-test generalisation.

Tune alert thresholds on validation data instead of using 0.5.